Informe técnico de pruebas locales

UNL GeoAsist · Backend de asistencia mediante QR

Validación técnica asistida por Codex

2026-10-20

Tabla de contenidos

1 Dictamen técnico 1
2 Alcance y entorno de ejecución 1
2.1 Control del consumo de recursos 2
3 Componentes funcionales evaluados 2
4 Resultados de la ejecución actual 3
4.1 Construcción y suites del backend 3
4.2 Comprobación HTTP del JAR en ejecución 4
4.3 Observaciones de ejecución 4
5 Evidencia anterior y límites pendientes 4
6 Reproducción local 5
6.1 Repetir la verificación del backend 5
6.2 Iniciar el JAR y comprobar HTTP 5
6.3 Regenerar el PDF 5
7 Identificación de los entregables 5

1 Dictamen técnico

El backend está disponible para pruebas locales con datos ficticios. En esta ejecución se aprobó la construcción del JAR y se completaron **60 pruebas en 23 suites**, con **cero fallos, cero errores y cero pruebas omitidas**. Después se inició el artefacto empaquetado y se aprobaron **siete comprobaciones HTTP reales** de autenticación, consulta y revocación de credenciales.

La API quedó atendiendo en **http://127.0.0.1:8080/api/v1**, exclusivamente en la interfaz local, con PostgreSQL en el puerto 5433. El perfil utilizado fue `dev` y sus identidades son ficticias. Las credenciales se cargaron desde el archivo local ignorado `.env`; este documento y las evidencias JSON no contienen contraseñas ni tokens.

Este dictamen confirma los escenarios ejecutados. No constituye autorización para usar datos reales ni una certificación de preparación integral para producción. No se midió cobertura porcentual de líneas o ramas: 60 pruebas aprobadas no significa cobertura del 100 %.

2 Alcance y entorno de ejecución

Componente	Configuración comprobada
Fecha y zona	20 de octubre de 2026; America/Guayaquil (UTC−05:00)
Sistema	Linux x86_64; equipo con aproximadamente 9,1 GiB de RAM
Java	Eclipse Temurin 21.0.12.1
Backend	Spring Boot 4.1.1; artefacto 0.1.0-SNAPSHOT
Persistencia	PostgreSQL 17.7; Flyway V001–V008

Componente	Configuración comprobada
Geografía opcional	Migración separada y comprobación de PostGIS
Pruebas	JUnit, Spring MockMvc y Testcontainers con PostgreSQL real
Construcción	Maven Wrapper; verificación de formato Spotless y fase `verify`
Informe	Quarto 1.10.19; motor Typst 0.15.1

La ejecución Maven comenzó alrededor de las **18:49** y terminó a las **18:51:19**. Maven registró **2 min** de tiempo total. Se ejecutó sobre la copia de trabajo actual, con compilación incremental; no se presenta esta ejecución como una construcción `clean` ni como un commit integrado. Los hashes de las fuentes permiten identificar lo verificado.

Las pruebas de integración crearon bases de datos aisladas mediante Testcontainers. La comprobación HTTP posterior utilizó la base local de desarrollo, con su semilla ficticia. En el arranque del JAR, Flyway validó las ocho migraciones base y confirmó que el esquema ya estaba en V008, sin requerir una migración adicional.

2.1 Control del consumo de recursos

Para evitar repetir la saturación observada durante Android, se ejecutó únicamente el backend: sin emulador, Gradle, Next.js ni banco de carga. Maven usó heap máximo de 384 MiB, un solo fork de pruebas y heap máximo de 768 MiB para ese fork. Se limitaron a dos los procesadores reconocidos por las JVM; las cien solicitudes de las pruebas de concurrencia se conservaron. La API empaquetada se inició posteriormente con heap máximo de 384 MiB.

Estos son límites de heap, no límites de memoria total del sistema. Metaspace, hilos, PostgreSQL y el IDE consumen memoria adicional. Una lectura durante las pruebas mostró aproximadamente 4,2 GiB de RAM disponible; esa lectura puntual no mide el consumo máximo.

3 Componentes funcionales evaluados

El backend separa actividades, sesiones, identidades, QR, asistencias, reportes y conservación. Las operaciones comunes utilizan autorización de servidor, validación de entrada, transacciones PostgreSQL, controles de idempotencia y auditoría.

Área	Comportamientos cubiertos por las suites
Identidad y ámbitos	Login/logout, sesión revocada, autorización por propietario, carrera y rol
Clases	Asistencia mediante cuenta de estudiante autorizado para la actividad
Eventos e invitados	Lista precargada; cédula sin autocaptura de nombres; cuentas existentes no eluden las credenciales
Tiempo y QR	Rotación, límites de vigencia y plazo de servidor exclusivo del formulario de invitados
Integridad	Una asistencia por persona/sesión, reintentos idempotentes y conflictos concurrentes
Cierre	Cierre definitivo y carreras entre cierre, envío y revisión
Geografía	OFF sin dependencia de PostGIS; reglas de precisión y revisión; extensión opcional
Reportes	Consulta/exportación por ámbito y separación entre lectura y escritura

Área	Comportamientos cubiertos por las suites
Auditoría	Registro de cambios y decisiones con responsable y motivo
Conservación	Cinco años calendario, anonimización, supresión y reintento tras fallos transaccionales

El plazo ampliado de invitados permanece separado de la vigencia ordinaria del QR. La identidad del invitado se contrasta con el listado cargado por el encargado; esa comprobación no demuestra por sí misma identidad física. La excepción de identidad documentada en ADR 0002 requiere revisión humana antes del uso real.

4 Resultados de la ejecución actual

4.1 Construcción y suites del backend

Maven terminó con **BUILD SUCCESS**. `spotless:check`, las pruebas y el empaquetado de Spring Boot se completaron correctamente. Los resultados siguientes proceden de los XML de Surefire de esta ejecución; los tiempos son por suite e incluyen preparación cuando corresponde.

Suite	Pruebas	Tiempo (s)
ActivityClosureTest	4	0.745
ActivityScopeTest	1	0.435
GuestConfigurationContractTest	3	1.102
AttendanceConcurrencyTest	6	67.043
AttendanceContractTest	2	0.432
GeographicPolicyTest	4	0.009
GeographyReviewContractTest	2	0.626
GuestDeadlineTest	4	1.001
LocationDisabledTest	1	0.276
ParticipantEligibilityTest	2	0.801
ReviewConcurrencyTest	4	0.562
TransientSecretCleanupTest	1	0.237
RetentionLifecycleTest	5	0.487
RetentionPolicyTest	1	0.003
AuthContractTest	7	0.556
QrLifecycleTest	1	20.969
AdministrativeAuditTest	2	0.479
ReportingAuthorizationTest	2	1.802
ReportingContractTest	2	0.352
OptionalPostgisTest	1	8.743
SessionContractTest	3	0.790
ArchitectureDependencyTest	1	0.019
PrivacyAndLimitsTest	1	2.170

En total: **60 pruebas; 0 fallos; 0 errores; 0 omitidas**. No se redujo el número de rondas para aligerar la ejecución. `AttendanceConcurrencyTest` incluye cien rondas independientes de cien solicitudes simultáneas para

una persona y sesión por ronda, comprobando exactamente una asistencia, además de otros cinco escenarios concurrentes. El caso constituye una de las 60 pruebas y produce 10.000 intentos en ese escenario; no son 10.000 pruebas JUnit.

La validación de límites de peticiones tiene una configuración propia con el multiplicador efectivo uno. Las suites funcionales permiten autenticaciones repetidas mediante configuración de laboratorio; esto no convierte sus resultados en una prueba de capacidad institucional.

4.2 Comprobación HTTP del JAR en ejecución

Las solicitudes siguientes viajaron por TCP hacia el servidor local, una vez empaquetado y arrancado el JAR. Complementan las pruebas MockMvc. La cuenta de prueba quedó con su token revocado al finalizar el recorrido.

Comprobación	HTTP esperado/recibido	Resultado
Actividades sin credencial	401/401	Aprobado
Login con contraseña incorrecta	401/401	Aprobado
Login DEV válido	200/200	Aprobado
Listado autenticado de actividades	200/200	Aprobado
Detalle de actividad autorizada	200/200	Aprobado
Logout	204/204	Aprobado
Credencial revocada rechazada	401/401	Aprobado

Los tiempos conservados en la evidencia JSON corresponden a solicitudes individuales de este recorrido. No son percentiles, una prueba de carga ni una medición del tiempo de escaneo con cámara.

4.3 Observaciones de ejecución

El arranque del JAR se completó en **12,287 s**, según su registro. Las suites emitieron advertencias de Spring sobre detección de clases de configuración de pruebas y su comportamiento en una versión futura. No provocaron fallos con las versiones verificadas; deben revisarse al actualizar el framework. No se modificó la configuración funcional para ocultarlas.

5 Evidencia anterior y límites pendientes

Los siguientes resultados están documentados previamente y **no se volvieron a ejecutar como parte de esta comprobación del backend**:

- Banco HTTP: 2.000 intentos en diez minutos, cincuenta personas/cinco sesiones con GPS OFF, cero errores y p95 de 487,77 ms. Es evidencia de laboratorio del entorno descrito en `rendimiento.md`, no un resultado nuevo de esta ejecución.
- Restauración de una copia ficticia en PostgreSQL aislado sin red, con aplicación de la política de conservación. No certifica copias, discos ni recuperación PITR institucional.
- Recorridos web y pruebas de componentes móviles. La compilación de un APK Android no sustituye su recorrido en un dispositivo; iOS también permanece pendiente.

Antes de declarar preparación para producción quedan pendientes la autenticación institucional real, la ejecución remota de CI, la revisión del equipo y las decisiones institucionales sobre identidad y tratamiento de datos. La aprobación automática geográfica debe permanecer bloqueada hasta contar con el ensayo real en aula y espacio abierto y su evidencia verificable. El tiempo escaneo→resultado requiere dispositivo y buena señal.

Esta ejecución tampoco es una auditoría de seguridad, una prueba de penetración ni una revisión exhaustiva de dependencias Java. No se aportan porcentajes de seguridad ni aprobación humana ficticia. Los avisos de dependencias de los clientes ya documentados siguen siendo un asunto separado antes del despliegue completo.

6 Reproducción local

6.1 Repetir la verificación del backend

Desde `services/api`, configurar Java 21 y ejecutar secuencialmente:

```
export MAVEN_OPTS='-Xmx384m -XX:ActiveProcessorCount=2'
./mvnw -B -T 1 spotless:check verify \
  -DforkCount=1 -DreuseForks=true \
  '-DargLine=-Xmx768m -XX:MaxMetaspaceSize=256m -XX:ActiveProcessorCount=2'
```

Docker debe estar disponible. Testcontainers utiliza imágenes PostgreSQL y PostGIS reales; si no están en caché, la descarga requiere red. Conservar el número completo de rondas de concurrencia para reproducir el alcance del resultado.

6.2 Iniciar el JAR y comprobar HTTP

Desde la raíz, con la base local activa y `.env` preparado:

```
python3 infra/with-local-env.py -- env \
  API_PORT=8080 SPRING_PROFILES_ACTIVE=dev \
  java -Xmx384m -XX:ActiveProcessorCount=2 \
  -jar services/api/target/geoasist-api-0.1.0-SNAPSHOT.jar \
  --server.address=127.0.0.1
```

Desde otra terminal en la raíz:

```
python3 infra/with-local-env.py -- \
  python3 infra/tests/backend-smoke.py \
  --output /tmp/backend-http.json
```

El script comprueba siete estados y escribe evidencia redactada. No crea actividades, sesiones académicas ni asistencias nuevas. Sí autentica una cuenta de desarrollo y revoca la credencial emitida. La API utiliza sus controles y trabajos de conservación configurados; no utilizar una base real para este procedimiento de desarrollo.

6.3 Regenerar el PDF

Se instaló Quarto en `~/.local/opt/quarto-1.10.19` y se creó el enlace `~/.local/bin/quarto`. Se verificó el SHA-256 del archivo oficial antes de extraerlo. Typst viene incluido; no se instaló LaTeX ni se requirió R/Jupyter.

```
quarto render docs/validation/informe-backend-local.qmd --to typst
```

El documento contiene resultados estáticos de esta ejecución: regenerar el PDF no vuelve a ejecutar las pruebas. Para actualizar el dictamen se deben repetir las verificaciones y actualizar sus evidencias.

7 Identificación de los entregables

Evidencia	Ubicación relativa a la raíz
Resumen y suites actuales	`docs/validation/backend-local-2026-10-20.json`
Estados HTTP redactados	`docs/validation/backend-http-2026-10-20.json`
Hash de cada archivo fuente	`docs/validation/backend-local-fuentes-2026-10-20.json`
Fuente editable del informe	`docs/validation/informe-backend-local.qmd`
Artefacto ejecutado	`services/api/target/geoasist-api-0.1.0-SNAPSHOT.jar`

Evidencia	Ubicación relativa a la raíz
Script HTTP reproducible	`infra/tests/backend-smoke.py`

SHA-256 del JAR ejecutado:

```
eec12a48b9768a471d5bb922baa4db59b57c0e2e8c60c686307bf369a00df877
```

La referencia Git de base es `82d3318688ab84eea71cecc5ebf8541f4b90f103`, con fuentes modificadas y archivos nuevos en la copia de trabajo. La referencia de base por sí sola no identifica todo el código probado; debe consultarse también el manifiesto de hashes. Los XML originales quedan en `services/api/target/surefire-reports`; no se incorporan cuerpos HTTP, secretos ni registros sin redactar al PDF.

Documentación de las herramientas: distribución oficial de Quarto y generación de PDF mediante Typst. Las afirmaciones funcionales se sustentan en las fuentes, los XML de pruebas y las evidencias locales identificadas en este informe. Revisión y aprobación humana: **pendientes**.